

Inhaltsverzeichnis

1. Architekturentscheidung: Auswahl des GUI-Frameworks.....	2
1.1. Zielsetzung.....	2
1.2. Definition und Gewichtung der Kriterien.....	2
1.3. Nutzwertanalyse.....	2
1.4. Ergebnis und Begründung der Entscheidung.....	4

1. Architekturentscheidung: Auswahl des GUI-Frameworks

1.1. Zielsetzung

Ziel dieses Abschnitts ist die Auswahl der geeignetsten Technologie für die grafische Benutzeroberfläche der Anwendung "Piep - Der Takt der Kasse". Da es sich um eine simulationslastige Anwendung mit Verwaltungsanteilen handelt, müssen sowohl dynamische grafische Darstellungen (bewegte Kunden, Karte) als auch klassische Formular-Elemente (Listen, Buttons) unterstützt werden.

1.2. Definition und Gewichtung der Kriterien

Um eine objektive Entscheidung zu gewährleisten, wurden sieben Qualitätsmerkmale definiert. Die Gewichtung erfolgt auf einer Skala von 1 (weniger wichtig) bis 5 (essenziell/k.o.-Kriterium).

- **Grafik- & Simulations-Fähigkeiten (Gewichtung: 5):** Maßgeblich für die performante Darstellung hunderter bewegter Objekte. Bewertet wird die Verfügbarkeit eines "Scene Graph" (Verwaltung von Objekten statt Pixeln).
- **Kompatibilität & Threading (Gewichtung: 5):** Verhindert das Einfrieren der GUI während der Simulation durch threadsichere Kommunikation.
- **Modernität & Widget-Umfang (Gewichtung: 4):** Sicherstellung eines professionellen "Look & Feel" durch native Steuerelemente (z. B. validierte Eingabefelder, Slider) für die Simulationssteuerung.
- **Ökosystem & Erweiterbarkeit (Gewichtung: 3):** Einfache Integration von Bibliotheken für Echtzeit-Graphen.
- **Dokumentation & Community (Gewichtung: 3):** Schnelle Problemlösung durch Verbreitung und offizielle Ressourcen.
- **Benutzerfreundlichkeit / UX (Gewichtung: 2):** Responsive Anpassung der Oberfläche an verschiedene Fenstergrößen.
- **Einarbeitungszeit (Gewichtung: 2):** Akzeptanz einer steileren Lernkurve zugunsten besserer Architektur.

1.3. Nutzwertanalyse

Der Vergleich umfasst die Standard-Bibliothek **Tkinter**, das NUI-Framework **Kivy** und das Industriestandard-Framework **PyQt6**. Bewertungsskala: **1 (mangelhaft)** bis **5 (sehr gut)**.

Kriterium	Gew.	Tkinter	Kivy	PyQt6	Fokus-Begründung
Grafik-Performance	5	2 Pixelbasiert	4 OpenGL, GPU-Last	5 Scene-Graph	Die QGraphicsView-Architektur verwaltet Objekte effizienter als reines Pixel-Zeichnen.
Threading	5	2 Blockierend	3 Umständlich	5 Signal-Slot	QThread entkoppelt Simulation und GUI sauber, verhindert "Einfrieren"
Widget-Umfang	4	2 Veraltet, Basic	3 Mobile-Fokus	5 Native Profi-Widgets	Professionelle Eingabeelemente und Layouts wirken in Qt nativ und modern, während Tkinter oft veraltet aussieht.
Ökosystem (Graphen)	3	3 Matplotlib	2 Multimedia	5 PyQtGraph	Nahtlose Integration von PyQtGraph für schnelle Echtzeit-Auswertungen.
Doku & Community	3	5 Sehr hoch	3 Mittel	4 Industriestandard	Exzellente Dokumentation und Industriestandard sichern schnelle Problemlösung.
UX / Responsivität	2	2 Starr	4 Flexibel	5 Layout-Manager	Layouts passen sich automatisch an die Fenstergröße an.
Einarbeitungszeit	2	5 sehr einfach	2 eigene Sprache	3 steile Kurve	Bessere OOP-Struktur rechtfertigt den höheren initialen Lernaufwand.
Gesamt-Nutzwert		66	74	113	PyQt6 gewinnt deutlich.

1.4. Ergebnis und Begründung der Entscheidung

Die Entscheidung fiel eindeutig auf **PyQt6**. Ausschlaggebend war neben der reinen Performance vor allem die architektonische Eignung für ein Schulprojekt mit Simulationscharakter.

1. **Architektur & OOP:** PyQt6 erzwingt durch sein objektorientiertes Design (z. B. Ableiten von QGraphicsItem für Kunden) eine saubere Softwarestruktur. Dies verhindert "Spaghetti-Code" und fördert das Verständnis für professionelle Softwareentwicklung.
2. **Performance (Scene Graph):** Die Anforderung, Kundenbewegungen flüssig zu animieren, wird durch die QGraphicsItem-Architektur von Qt optimal erfüllt. Im Gegensatz zu Tkinter, das oft den gesamten Bildbereich neu zeichnen muss ("Redraw"), nutzt Qt einen optimierten Szenengraphen. Das bedeutet, das Framework "weiß", wo sich Objekte befinden und berechnet nur die pixelgenauen Änderungen, was CPU-Ressourcen spart.
3. **Stabilität (Threading):** Durch die saubere Trennung von GUI und Logik mittels QThread und Signalen wird sichergestellt, dass die Oberfläche auch bei intensiven Berechnungen reaktionsfähig bleibt ("Thread-Safety").